

Designing Neural Simulation Pipeline

Technical report

Karol Chlasta¹², Paweł Sochaczewski¹ and Grzegorz M. Wójcik¹³

¹ Department of Computer Science
Polish-Japanese Academy of Information Technology, Warsaw, Poland
{chlasta,gmwojcik}@pja.edu.pl

² Department of Management in Networked and Digital Societies
Kozminski University, Warsaw, Poland
{kchlasta,psochaczewski}@kozminski.edu.pl

³ Department of Neuroinformatics and Biomedical Engineering
Maria Curie-Skłodowska University in Lublin, Poland
gmwojcik@live.umcs.edu.pl

We present Neural Simulation Pipeline (NSP), a set of 5 user, and 12 container Bash scripts to facilitate the large scale neural simulations and their deployment to multiple compute infrastructures using Docker. We evaluate our pipeline by performing 170 simulations the HPI Future SOC Lab cluster. We compare the standard and containerised execution, and we present the results.

1 Introduction

Large-scale simulations of biologically realistic neural networks require large and expensive computational resources. Liquid State Machines [8] are important in brain modelling and increasingly important in different engineering applications [12].

The spiking neural networks built of Hodgkin-Huxley (HH) [6] neurons behave like Liquid State Machines (LSM) [7]. These neural simulations of LSMs can be performed using GENESIS simulation engine [2].

Neural simulation environments are often complex and expensive to build and maintain. Their configuration and model deployment might present a significant barrier for many researchers tackling biocybernetic modelling. This is because building, testing and deploying cybernetic models often requires different software libraries as dependencies, and the ability to consume significant amounts of parallel or distributed computing power to speed up the long running computations. The matter is especially important for developing larger, and more complex scripts simulating elements of human brain. The task is not trivial from technical perspective, even when using a well established simulation engine [4].

Authors believe that the problem of building, testing and deploying cybernetic models, as well as their different software dependencies could be resolved using Docker [9]. Docker is the most popular container platform; as suggested in the recent IDC's white paper [3], it has already attracted "a significant amount of industry recognition", and it has the opportunity to "define the road map for all container platforms". It also has a potential to become a key component of "all the enterprise IT environments globally", due to its operating system neutrality. The

same report mentions that “the container revolution is under way”, with a forecast of 1.8 billion enterprise container instances deployed by 2021.

As summarised by Nickoloff and Kuenzlin [10], Docker platform uses the low-level kernel internals to run applications in containers using the *Docker Engine*. The architecture of Docker containers relies on both *namespaces* and *control groups*. The process is transparent for the applications, as a container is a ring-fenced area of operating system with limits imposed on how much system resource it can use. Historically the low-level Linux kernel constructs were available in Linux out of the box, but they were difficult to use. This gap was explored by the Docker Engine, whose authors have facilitated the usage of the low-level kernel constructs. The Docker engine was designed in a modular way, with two main interfaces: the Command Line Interface (CLI) and Application Programming Interface (API). The engine creates a layer of abstraction for all the required kernel internals, and creates a container that is designed for hosting specific applications and their dependencies [9].

Although the containers can be deployed and managed manually, most organisations use different continuous integration/continuous delivery (CI/CD) or DevOps pipelines to automate the process [5]. In this paper we present Neural Simulation Pipeline (NSP), that was developed with Bash [11].

1.1 Project Idea

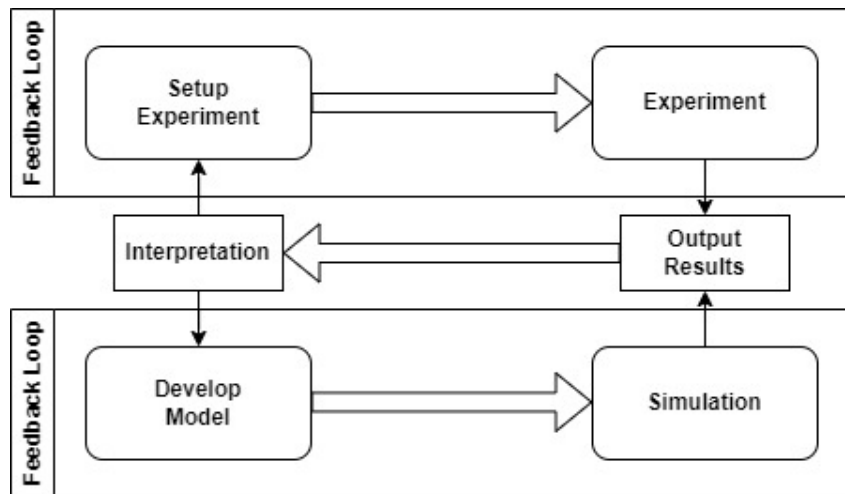


Figure 1: The relationship between experiment and simulation in computational neuroscience. Own figure, based on GENESIS documentation [2].

Authors claim that numerical simulations must integrate a robust model development methodology with adequate testing and simulation steering workflows to increase scientific throughput and improve utilisation of current and next-generation

computational infrastructure available both on-premise and in-cloud. To this end, there is the need to transform the end-to-end simulation development and execution workflow from the one that is non-universal and manual to one that is standardised and automated.

Fig. 1 presents the relationship between experiment setup and simulation run in computational neuroscience. As it can be seen, *conducting experiment* and *running simulation* are two distinct iterative loops connected by a feedback process. This process uses *interpretation of output results* to *design new experiment setups* and *develop new cybernetic models*.

GENESIS supports the lower loop within the system as shown above, but it leaves a gap for *setting up and executing the experiments* (e.g. setting up model parameter values, different stimulus, storing the parameters and providing execution statistics). A similar gap was identified for other simulation engines e.g. for Nengo [1]), what triggered the idea of creating a more universal simulation pipeline called *Neural Simulation Pipeline* (NSP).

2 Project Activities

2.1 Simulating Visual System

The LSM-based cybernetic model of visual systems used in our simulations had already been presented during the 24th HPI Future SOC Lab Day on April 26th, 2022 as “Exploring Neural Columns for Real-Time Information Processing”. That report proposed a modular, bio-inspired model built with two main components, an *Input* (acting as a retina of the system) and *Liquid* (acting as a visual cortex, built of a single LSM column).

Our Hodgkin-Huxley Liquid State Machine (HHLSM) model uses high fidelity multi-compartmental neurons with voltage-activated sodium and potassium channels. The structure of each column in the models is the same. Each HHLSM was build in 6 versions:

1. 1040 neural cells placed in a rectangular cuboid of $8 \times 5 \times 25$.
2. 2040 neural cells placed in a rectangular cuboid of $8 \times 5 \times 50$.
3. 3040 neural cells placed in a rectangular cuboid of $8 \times 5 \times 75$.
4. 4040 neural cells placed in a rectangular cuboid of $8 \times 5 \times 100$.
5. 8040 neural cells placed in a rectangular cuboid of $8 \times 5 \times 200$.
6. 12040 neural cells placed in a rectangular cuboid of $8 \times 5 \times 300$.

The research in the latter part of this year (2022) focused mostly on the new, larger Type 5 and 6 models. Although NSP allows for a dynamic change of the size of the model, we have standardised the column for this paper to the 6 above sizes. In the simulated task, we provide (in the automated way) each model with 3 different

stimulation patterns of “o”, “A”, “1”. This gives us an opportunity to evaluate the LSM system built of an increasing number of neurons in a standardised way.

2.2 Architecture of Neural Simulations Pipeline

The NSP is based on a set of services that provide model developers with testing and deployment automation using a set of Bash scripts. The Pipeline’s source code is stored in *nsp-code* repository available at <https://github.com/KarolChlasta/nsp-code.git>. This repository is used for all the container builds, and it contains the *Dockerfile* describing the automated build process (in *nsp-server/Dockerfile*). The other repository used in the project is called *nsp-model*. It stores the source code of all the cybernetics modes used in the simulations.

We have built our NSP image for GENESIS simulator using the official Canonical Ubuntu bionic (version bionic-2022101) from https://hub.docker.com/_/ubuntu18.04. The automated build process installs: *csch*, *g++*, *libxt-dev*, *libxt6*, *libxtst6*, *libxtst-dev*, *libxmu-dev*, *mpich*, *gcc*, *bison*, *flex*, *libncurses5-dev*, *libxt-dev*. As a result the pipeline, both GENESIS and its parallel version PGENESIS are compiled with all the dependencies and yje official NSP image can be found at: <https://hub.docker.com/r/karolchlasta/genesis-sim/tags>. The image uses 424.83 MB and can be pulled from DockerHub using the below command:

```
1 $ docker pull karolchlasta/genesis-sim:prod
```

The architecture of NSP allows to setup a pipeline on multiple compute platforms. The following diagram (Fig. 2) exhibits a schematic view of the pipeline, with stages describing typical steps needed in the development workflow (e.g. for cybernetic model development, or for adding any new simulation engines). NSP allows for the configuration of multiple simulators. In its current version it supports two (parallel and standard) GENESIS simulation engines only, but adding a new one only requires analogous changes to the *Dockerfile* in the *nsp-code* repository. The process of building such a pipeline is described in Fig. 2. The top part of the diagram focuses on the overall development process, whereas the bottom on the experiment workflow. The development process is two-fold. If the experiment requires to set up another simulation engine, one might need to change the pipeline’s code (*nsp-code* repository). In the most common model development case (for a model using the existing simulation engine), it is sufficient to change the model scripts in the artefact store, the *nsp-model* repository is synchronised with the artefact store.

The Bash scripts that were developed for the NSP are presented in Table 1. NSP facilitates the automated testing of the cybernetic models through *runUnitTest.sh* and *runUnitTestCheck.sh*. These files contain sample tests. If a new model is developed, than new test scripts might need to be created in an analogous way. Ideally the model will have a full test coverage designed by a model developer. This gives confidence that a model is tested, and any bug is identified early in the development process. This is especially important for long running brain simulation models, whose bugs could often only be identified post-hoc e.g. after running for several days (or weeks) on expensive supercomputers. Similarly to software

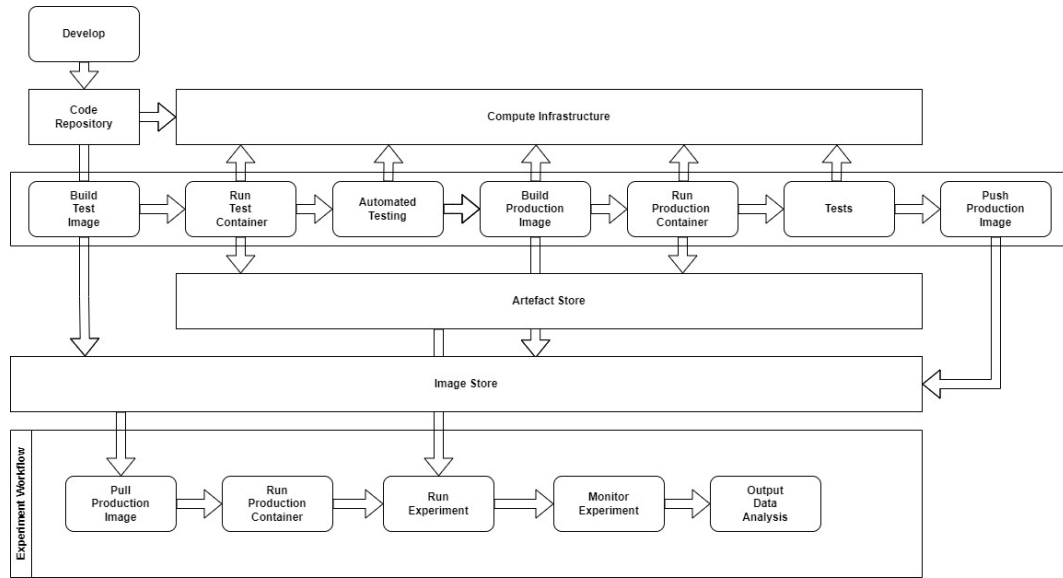


Figure 2: Schematic view of Neural Simulations Pipeline.

Table 1: Full list of scripts in Neural Simulations Pipeline.

User Scripts	Container Scripts
buildNspServerImage.sh	saveStat.sh
pushNspServerImage.sh	configAWSCLI.sh
getAWScredencials.ps1	downloadExp.sh
loadModels.sh	listExp.sh
runExperimentsManager.sh	runExp.sh
	showStat.sh
	showSystemInfo.sh
	validatePositiveInteger.sh
	validateRealNumber.sh
	validateRange.sh
	startNspContainer.sh
	pullNspImage.sh

development, programming cybernetic models also requires bug fixing. The sooner a bug can be identified in the model script, the less expensive the resolution is, because the model developer needs less time to get familiar with a given fragment of the code. That is where the benefits of NSP testing are especially visible.

There are also other scripts supporting experiment setup and execution. These are three scripts to facilitate parameter validation. These are *validateRange.sh*, *validatePositiveInteger.sh*, and *validateRealNumber.sh*.

The current version of NSP also allows for a choice of simulation engine. We have tested the pipeline using both standard and parallel GENESIS simulation engines. The simulator is selected via the parameter of *runExp.sh* script with flag *parallelMode = 1* for a parallel run.

The pipeline also allows to define and reuse certain variables that are universal and independent from the simulation engine. We call them *NSP variables*. These variables should be added to the model's source code between the special character of "\$ \$" (e.g. "\$nspVariableName\$"). As a result the models' code can be more standardised, even across different simulation engines. The current version of the pipeline recognises 12 NSP variables:

1. \$modelName\$
2. \$experimentSuffix\$
3. \$experimentDesc\$
4. \$simulationTimeStepInSec\$
5. \$simulationTime\$
6. \$columnDepth\$
7. \$synapticProbability\$
8. \$retX\$
9. \$retY\$
10. \$parallelMode\$
11. \$numNodes\$
12. \$modelInput\$

2.3 Future SOC Lab Resources

The main computing resource that we received from Future SOC Lab and used in the project was an Ubuntu 18.04.5 bionic virtual machine (vm-20211005-001.fsoc.hpi.uni-potsdam.de, running Linux 4.15.0-143-generic on x86_64 architecture). It was used as our development workstation. We received a privileged access to the machine allowing us to compile and install all the required software e.g. the latest version of

GENESIS simulation environment [2] (version 2.4 from May 2019) and Docker [9] platform (version 20.10.08 from July 2021). The machine we received was equipped with 8 CPUs: 8 x 2.00 GHz, memory (RAM): 7.79 GB, local disk: we used 63.1% of 69.38GB as of November 20th, 2022. We also used the Docker Registry at HPI at `registry.fsoc.hpi.uni-potsdam.de`.

2.4 Limitations

There are a few limitations in the current version of the Neural Simulations Pipeline. Firstly, the current version of our official NSP Docker image with the last version of GENESIS simulation engines is relatively large. It requires 1.17 GB in the local repository, and 424.83 MB in the remote registry (that is after compression, at <https://hub.docker.com/r/karolchlasta/genesis-sim>). We believe that the image could be optimised by removing some uncritical operating system tools and utilities.

Secondly, the current Neural Simulations Pipeline supports GENESIS and PGENESIS [2] only. We would also like to create a new image allowing us to run other (e.g. functional) simulators like Nengo [1].

Thirdly, although we are using the latest, official version of GENESIS engine [4], we are still encountering some issues that limit neural network scalability. We were unable to successfully simulate the RetNet(8x5,1,300) model with 12040 neurons in the repeatable manner. It seems that we might need to experiment with a garbage collector to reduce the memory usage, if we want to simulate larger models (in our case consuming over 5GB of memory) without any run-time errors and integer overflow issues.

Finally, we would like to redesign the NSP to provide a service for multiple research teams at the same time, and enable interdisciplinary work between different profiles of researchers. That would likely require a web-interface for experiment management, as well as and security model based on a defined set of access rules e.g. for model developers and/or neuroscientists.

3 Findings

We show that our LSM models react differently to 3 different input patterns that are numbers (0, 1) and letter (A). We performed the additional 170 simulations using 6 types of the RetNet models. We measured the model execution time (CPU Time), memory consumption as well as the number of spikes in each simulation run. The aggregated results of the simulations are presented in Table 2.

We have also compared the standard and containerised simulation setup. The results measured on the HPI cluster do not indicate any major negative impacts of containerisation on the overall simulation performance. The average time (CPU time) needed to complete the containerised simulations of our RetNet model is 125.8 % of the average simulation time needed to complete the same simulation on the virtual machine. Interestingly, the opposite was measured for memory

consumption, the containerised simulation consumed only 92.6 % of the memory needed for a standard execution. Although these results require further validation, we believe that the overall performance overhead of containerised execution might still be acceptable for computationally intensive neural simulations.

4 Next Steps

In the upcoming months we would like to finalise our research to run larger models, and present a more in-depth evaluation of NSP.

As the current solution still requires basic knowledge of the operating system shell, running Bash scripts (and working knowledge of some AWS cloud services), we would like to create a web application providing a simulation service using NSP containers without the need for explicitly running the scripts. This would allow us to expose NSP as a Scientific Workflow Management System to a wider community, gather feedback, and potentially also allow us to perform more extensive testing with different public cloud providers. That could lead to enhancing the cloud service to facilitate planning and forecasting of costs for large-scale simulations. We imagine, a trial run in a public cloud could help computational neuroscience researchers with cost estimation. An automated trial run of a smaller model could be a good proxy for a full scale execution, and it would allow them to budget for computations with NSP more easily and accurately; apart from just providing the researchers the experiment management and execution capabilities.

As a result, the practical significance of NSP would be than in reducing entry barriers to numerical systems modelling and large-scale simulations through a Docker-based pipeline, that could be executed across multiple compute infrastructures.

5 Acknowledgements

We thank Michael Connolly for proofreading.

6 Appendix

6.0.1 Neural Simulation Pipeline User Scripts

1. *buildNspServerImage.sh* - Bash script that clones the repository with the source code of cybernetic models (nsp-model) and builds Docker image of a simulator from a DockerHub registry (karolchlasta\genesis-sim) by compiling the simulation software with all the required libraries (from the 'main' branch) and building the container with the input tag name e.g. 'prod' (production) tag. The script generates locally usable container image with GENESIS software,

Table 2: Neural Simulation Pipeline - Standard vs Containerised Execution on HPI infrastructure.

Standard Execution					
Model	Neurons	Pattern	Spikes	Execution [s]	Memory [MB]
RetNet(8x5,1,25)	1040	0	20829	132.27	18.78
		A	20827	136.13	37.97
		1	20818	124.69	21.25
RetNet(8x5,1,50)	2040	0	61191	239.34	44.56
		A	61236	205.75	52.51
		1	61234	231.47	49.77
RetNet(8x5,1,75)	3040	0	121605	628.24	53.28
		A	121646	536.32	65.27
		1	121627	648.69	53.42
RetNet(8x5,1,100)	4040	0	202045	1269.55	88.81
		A	202036	1269.12	94.99
		1	202056	1321.30	72.24
RetNet(8x5,1,200)	8040	0	6512454	34082.88	1766.86
		A	6512440	34065.61	1791.56
		1	6568685	26548.51	1760.26
Containerised Execution					
Model	Neurons	Pattern	Spikes	Execution [s]	Memory [MB]
RetNet(8x5,1,25)	1040	0	20834	45.66	26.37
		A	114427	185.05	46.51
		1	114403	214.94	71
RetNet(8x5,1,50)	2040	0	61215	150.40	57.97
		A	428368	1679.33	150.17
		1	428500	1659.42	146.53
RetNet(8x5,1,75)	3040	0	121659	495.38	63.01
		A	942366	3949.42	292.29
		1	942452	3957.19	80.70
RetNet(8x5,1,100)	4040	0	202014	938.60	73.91
		A	1656397	7253.15	139.56
		1	1656444	7467.51	437.87
RetNet(8x5,1,200)	8040	0	6512411	33000.14	1766.57
		A	6512452	33434.70	1766.35
		1	6512401	33198.74	437.87
RetNet(8x5,1,300)	12040	0	14568503	87163.24	4745.92

models and build logs. The build logs are named using the ID of the build process (unique) e.g. `docker_build17048.log`.

- *Inputs:*
 - a) Tag name for container: prod, test.
 - b) Login credentials to a remote Git repository with the source code of cybernetic model(s); only needed if the repository is not public. Currently: password to the nsp-model repository.
 - *Outputs:*
 - a) Container image in the local repository of images that is ready to run.
 - b) Build logs in folder `docker_build_logs`.
2. *pushNspServerImage.sh* - Bash script that pushes the tested image to the remote Docker registry (`karolchlasta\genesis-sim:input_tag_name`).
- *Inputs:* Tag name of the local image to push to remote Docker registry.
 - *Outputs:*
 - a) Container production image in the remote Docker registry (repository) of images.
 - b) Build logs in folder `docker_build_logs`.
3. *getAWSCredentials.ps1* - PowerShell script that gets AWS session token. Returns AWS (CLI) credentials. Might be useful if the development machine runs on Windows.
- *Inputs:* N/A.
 - *Outputs:*
 - a) AWS AccessKeyId.
 - b) AWS SecretAccessKey.
 - c) AWS SessionToken.
4. *loadModels.sh* - Bash script that downloads the content of the repository with models and uploads them to NSP artefact store (AWS S3).
- *Inputs:* N/A.
 - *Outputs:*
 - a) Remote code repository copied to the NSP's artefact store (AWS S3).
5. *runExperimentsManager.sh* - bash script for running batch experiments simulations. Script check every 15 minutes if simulator is working, if not executes next experiment simulations command from the *experimentsToRun.txt* file.
- *Inputs:*
 - a) File *experimentsToRun.txt* with commands for simulator.

- *Outputs:*
 - a) Experiment results uploaded to AWS Simple Cloud Storage (S3).
 - b) File *historyOfExperiments.txt* with history of batch simulations commands.

6.0.2 Neural Simulation Pipeline Container Scripts

1. *saveStat.sh* - A Bash helper script that saves the global simulation statistics into the AWS S3 object. The script takes the simulator's output (.out file), saves it as a single row and transfers the file to the global stats file in the AWS public cloud (s3://genesis-mods/globalStatistics.txt).

- *Inputs:*
 - a) Model name and parameters.
- *Outputs:*
 - a) Updated globalStatistics.txt file.

2. *configAWSCLI.sh* - Bash script to configure AWS credentials. These can normally be found in user folder (.aws\credentials), a single AWS configuration is needed per Region. This script might be useful if the development machine runs on Windows.

Output: .

- *Inputs:*
 - a) AWS AccessKeyId.
 - b) AWS SecretAccessKey.
 - c) AWS SessionToken.
- *Outputs:*
 - a) Config files in user folder (.aws\credentials).

3. *downloadExp.sh* - Bash script to create a local copy (e.g. on development workstation) of all the data for a given experiment stored in NSP's cloud artefact store (AWS S3). Authorisation to cloud resources happens via *configAWSCLI.sh*. All other NSP scripts use this script to download from the NSP artefact store.

- *Inputs:*
 - a) AWS S3 URI e.g. (s3://nsp-mods/2022-10-16-182851_2neurons_test).
- *Outputs:*
 - a) A folder with all the data for a given experiment.

4. *listExp.sh* - Bash script to show what experiments are currently loaded to NSP's artefact store. Experiment is defined as a folder that contains all the experiment parameters, model source code, outputs (all data files) as well as logs and statistics.

- *Inputs:* N/A.
- *Outputs:*
 - a) Experiment names written to a standard output.

5. *runExp.sh* - Bash script that runs an experiment. It specifically:

- a) Downloads the model (sources) from the artefact store (AWS S3).
- b) Reads and validates experiment parameters from the user, sets the default parameters.
- c) Parses the model code for the NSP parameters. The original model file is saved as an .org file.
- d) Builds the experiment name using the concatenated NSP parameters of date (%Y-%m-%d-%T), modelName, retX, retY, columnDepth, modelInput, simulationTime, experimentSuffix e.g. 2022-11-14-143707_RetNet40_5x8_75_o_1_HPIDock
- e) Runs the selected simulation engine (e.g. standard or parallel version).
- f) Prints the experiment execution status to the standard output.
- g) Saves environment statistics to the global statistics file.
- h) Saves the simulation files to the experiment folder in AWS S3.
- i) Saves the experiment time (start and finish).
- j) Runs *showSystemInfo.sh*, and generates *experimentInfo.out* that contains all the parameters, start/stop time and environment statistics, as well as the screen output from the time of the experiment, logs, and the errors (ModelName.err with errors).

- *Inputs:*
 - a) Model Name.
 - b) Experiment Suffix (Env).
 - c) Experiment Description.
 - d) Simulation Time Step [s].
 - e) Simulation Time [s].
 - f) LSM Column Depth.
 - g) synapticProbability.
 - h) retX.
 - i) retY.
 - j) parallelMode.
 - k) numNodes.
 - l) modelInput.
- *Outputs:*
 - a) experimentInfo.out

- b) `ModelName.out`
 - c) `ModelName.err`
 - d) All the relevant `.dat` files, as defined in the model.
6. *showStat.sh* - Bash script that prints % of experiment execution based on a defined simulation step. Uses *grep* on `.dat` files.
- *Inputs*: N/A.
 - *Outputs*:
 - a) Text sent to standard output.
7. *showSystemInfo.sh* - Prints the predefined OS statistics incl. information on hardware, kernel, memory, network, block devices, file systems, available space and active processes.
- *Inputs*: N/A.
 - *Outputs*:
 - a) Text sent to standard output.
8. *validatePositiveInteger.sh* - Checks if the parameter is positive Integer
- *Inputs*: `parameterForCheck`.
 - *Outputs*:
 - a) Text sent to standard output.
 - b) Return 0, if check pass; return grater than zero if check fails.
9. *validateRealNumber.sh* - Checks if the parameter is Real number
- *Inputs*: `parameterForCheck`.
 - *Outputs*:
 - a) Text sent to standard output.
 - b) Return 0, if check pass; return grater than zero if check fails.
10. *validateRange.sh* - Checks if the parameter is inside the range.
- *Inputs*:
 - a) `parameterForCheck`.
 - b) `beginOfRange`
 - c) `endOfRange`
 - *Outputs*:
 - a) Text sent to standard output.
 - b) Return zero, if check pass; return grater than zero if check fails.
11. *startNspContainer.sh* - Starts an NSP container based on the image with of a given tag.

- *Inputs:*
 - a) tag name (default 'prod').
 - *Outputs:*
 - a) running container.
12. *pullNspImage.sh* - Pulls the NSP remote image to the local Docker repository.
- *Inputs:*
 - a) image tag name (default karolchlasta/genesis-sim:\$TAG_NAME)
 - *Outputs:*
 - a) image in the local Docker repository.

6.1 Running Official NSP Container with GENESIS

1. Clone NSP source code repository.
2. Build docker production container from the official NSP image with the 'prod' tag.
3. Run standard short experiment for testing.
4. On successful test result, perform local NSP container image deployment. push the docker NSP container image from the local images repository (with 'prod' tag) to the DockerHub repository.

6.2 Preparation for Model Development

1. Create an account in dockerHub registry.
2. Install docker engine from <https://docs.docker.com/engine/install/>.
3. Log into local docker registry.
4. Create user account in public cloud for accessing NSP results.
5. Create technical user account with limited privileges read/write access to s3 only (used by NSP to get and save data from s3).
6. Install aws cli.
7. Setup aws cli.
8. Clone NSP source code repository.
9. Clone NSP Model source code repository.
10. run container

6.3 NSP Cybernetic Model - Development

1. Construct cybernetic model by preparing its secure code.
2. Add NSP model variables. Sample code with the NSP mole variables

```
1 ...
2 float dt = $simulationTimeStepInSec$ // simulation time step in sec
3 ...
4 make_circuit_3d /cell /column_net_1 $retX$ $retY$ $columnDepth$
5 make_circuit_3d_output /column_net_1 $retX$ $retY$ $columnDepth$
   $modelName$-$modelInput$-column
6 ...
7 elif ( $modelInput$ == 13 ) //T
8   make_synapse /input /retina_net_1_1/dend/Ex_channel 2 0
9   make_synapse /input /retina_net_2_1/dend/Ex_channel 2 0
10  make_synapse /input /retina_net_3_1/dend/Ex_channel 2 0
11  ...
12  echo Pattern T
13  ...
14  // start simulation
15  step $simulationTime$ -time
16  ...
```

3. Add a new model to repository (nsp-model).
4. Clone the NSP model code to a development workstation (Windows or Linux) in order to load models from the repo to AWS S3.

```
1 $ git clone \url{https://github.com/KarolChlasta/nsp-model.git}
```

5. Load models from repository to s3 repository (loadModels.sh).

```
1 $ cd nsp-model
2 $ ./loadModels.sh
```

6. Run an experiment selecting a new model (through the process runExperiment p in model).

7. check simulation output and location results.

```
1 $ aws s3 ls s3://genesis-mods
2
3     PRE 2022-11-08-083003_RetNet40_5x8_50_T_1_HPI-Docker/
4     PRE 2022-11-08-091505_RetNet40_5x8_50_K_1_HPI-Docker/
5     PRE 2022-11-08-164749_RetNet40_5x8_50_J_1_HPI-Docker/
6     PRE 2022-11-08-190216_RetNet40_5x8_3_A_1_HPI-Docker/
7     PRE 2022-11-08-190931_RetNet40_5x8_500_0_1_HPI-Docker/
8     PRE 2022-11-09-150043_RetNet40_5x8_13_0_1_HPI-Docker/
9     PRE 2022-11-09-151850_RetNet40_5x8_13_0_1_Windows11-8GB/
10    PRE 2022-11-09-215406_RetNet40_5x8_200_0_1_HPI-Docker/
11    PRE 2022-11-10-075753_RetNet40_5x8_200_1_1_HPI-Docker/
12    globalStatistics.txt
```



```

1 $ aws s3 ls s3://nsp-project/experiments/2022-11-09-150043
   _RetNet40_5x8_13_0_1_HPI-Docker/model/
2 2022-11-09 16:00:57      5290 RetNet40-old.g
3 2022-11-09 16:00:59     16788 RetNet40.g
4 2022-11-09 16:00:56     17263 RetNet40.g.org
5 2022-11-09 16:00:55      1088 cell.p
6 2022-11-09 16:00:58     14443 experimentInfo.out
7 2022-11-09 16:00:55      1961 functions.g
8 2022-11-09 16:00:54     13575 patterns.g
9 2022-11-09 16:00:53      1559 protodefs.g
10 2022-11-09 16:00:52      1249 start.sh
11
12 $ aws s3 ls s3://nsp-project/experiments/2022-11-09-150043
   _RetNet40_5x8_13_0_1_HPI-Docker/results/
13 2022-11-09 16:02:10     721644 RetNet40-0-column.dat
14 2022-11-09 16:02:11     35200 RetNet40-0-retina.dat
15 2022-11-09 16:02:12        250 RetNet40.err
16 2022-11-09 16:02:13    2283266 RetNet40.out
17 2022-11-09 16:02:15     14510 experimentInfo.out

```

6.4 Running Single Experiment

1. Construct the model source files for Genesis simulator.
2. Add NSP parameters to the model.
3. Add model source code to model repository. The name of the model should not contains special characters like [:].

4. clone NSP repository

```
1 $ git clone \url{https://github.com/KarolChlasta/nsp-model.git}
```

5. Upload all model source code into AWS S3 repository by executing script inside repository:

```
1 $ ./uploadModelsS3.sh
```

6. Start the container instance:

```
1 $ docker run -it karolchlasta/genesis-sim:prod
```

7. Get the name of working NSP container:

```

1 $ docker ps
2
3 CONTAINER ID IMAGE                                NAMES
4 fde8447cad99 karolchlasta/genesis-sim:prod jovial_fermi

```

8. Sample execution of NSP experiment run command for Linux operating system is:

```
1 $ docker exec jovial_fermi bash -c ' /usr/local/nsp-server/src/sh/  
setEnv.sh; /usr/local/nsp-server/src/sh/runExp.sh RetNet40 HPI-  
Docker RetNet40 0.00005 1 500 0.1 5 8 0 1 A; ';
```

9. Sample execution of NSP experiment run command for Windows operating system is:

```
1 $ docker exec jovial_fermi bash -c " /usr/local/nsp-server/src/sh/  
setEnv.sh; /usr/local/nsp-server/src/sh/runExp.sh RetNet40  
Windows-11-8GB-Docker RetNet40 0.00005 1 13 0.1 5 8 0 1 0 ";
```

6.5 Running Experiments in Batch Mode

1. Start the container instance:

```
1 $ docker run -it karolchlasta/genesis-sim:prod
```

2. Get the name of the NSP container:

```
1 $ docker ps  
2  
3 CONTAINER ID   IMAGE                                NAMES  
4 fde8447cad99   karolchlasta/genesis-sim:prod       jovial\_fermi
```

3. Fill the file *experimentsToRun.txt* with the desired experiment commands. A sample line for Linux is:

```
1 docker exec jovial\_fermi bash -c ' /usr/local/nsp-server/src/sh/  
setEnv.sh; /usr/local/nsp-server/src/sh/runExp.sh RetNet40 HPI-  
Docker RetNet40 0.00005 1 500 0.1 5 8 0 1 A; ';
```

4. Run the script *runExperimentsManager.sh* to start the process of running experiments in a batch mode (so one after the another). This is especially useful for long running simulations.

6.6 Pipeline Preparation Stage

1. Docker engine installation:

- a) Install docker engine from <https://docs.docker.com/engine/install/>.
- b) Log into local docker registry.

```
1 $ docker login userNameAccount
```

2. NSP setup:

- a) Create user account in public cloud for accessing results from NSP.
- b) Create technical user account with limited privileges read/write access to s3 only (used by NSP to get and save data from s3).
- c) Install aws cli.

- d) Setup aws cli.
- e) Clone NSP source code repository.
- f) Clone NSP Model source code repository.

3. Starting Container with a simulator:

```
1 $ docker run -it karolchlasta/genesis-sim:prod
```

6.7 Pipeline Setup Stage

1. Select model for experiment

2. Starting Model simulation

- a) Get the name of working NSP container:

```
1 $ docker ps
2
3 CONTAINER ID IMAGE NAMES
4 fde8447cad99 karolchlasta/genesis-sim:prod jovial_fermi
```

- b) Sample execution of NSP experiment run command for Linux operating system is:

```
1 $ docker exec jovial_fermi bash -c ' /usr/local/nsp-server/src/
sh/setEnv.sh; /usr/local/nsp-server/src/sh/runExp.sh
RetNet40 HPI-Docker RetNet40 0.00005 1 500 0.1 5 8 0 1 A; '
```

- c) Sample execution of NSP experiment run command for Windows operating system is:

```
1 $ docker exec jovial_fermi bash -c " /usr/local/nsp-server/src/
sh/setEnv.sh; /usr/local/nsp-server/src/sh/runExp.sh
RetNet40 Windows-11-8GB-Docker RetNet40 0.00005 1 13 0.1 5 8
0 1 0 ";
```

the parameters from above commands would be like below:

```
1 Parameters:
2 1. modelName = RetNet40
3 2. experimentSuffix = Windows11-8GB
4 3. experimentDesc = RetNet40
5 4. simulationTimeStepInSec = 0.00005
6 5. simulationTime = 1
7 6. columnDepth = 13
8 7. synapticProbability = 0.1
9 8. retX = 5
10 9. retY = 8
11 10. parallelMode = 0
12 11. numNodes = 1
13 12. modelInput = 0 (0)
```

6.8 Pipeline Execution Stage

1. Parameters validation. NSP contains following validation checks for input parameters:

- a) positive Integer
- b) inside the numerical range
- c) greater than value
- d) less than value

2. Saving input parameters to results repository

3. Get the name of working NSP container:

```
1 $ docker ps
2 CONTAINER ID   IMAGE                                NAMES
3 fde8447cad99   karolchlasta/genesis-sim:prod       jovial_fermi
```

4. Selecting simulation engine. NSP chose between Genesis and Pgenesis simulators

5. Experiment execution.

6. Experiment monitoring. Scientist can get information about status of experiment execution from many sources depending of need:

- a) Analyse output files from cloud Artefact's store, accessed via Web browser Cloud console or command line interface.
- b) Direct monitoring of the machine on which the NSP container is run.
- c) Direct monitoring of the NSP container.

- a) Entering container

```
1 $ docker exec -it jovial_fermi bash
```

- b) Checking the progress of simulation

```
1 $ cd /usr/local/genesis-2.4/simulationFolder
2 $ tail -n 10 *.dat
3 -rw-r--r-- 1 root root 440 Nov 10 08:13 RetNet40-1-retina.dat
4 -rw-r--r-- 1 root root 818 Nov 10 08:14 RetNet40-1-column.dat
5 RetNet40-1-retina.dat:/retina_net_4_8/soma/spike 0.004300
6 RetNet40-1-column.dat:/column_net_1_5_8_195/soma/spike 0.006300
```

7. Saving experiments results

```
1 $ aws s3 copy . s3://genesis-mods/globalStatistics.txt
```

8. Saving experiments stats or Errors (saveStat.sh)

```
1 $ aws s3 copy . s3://genesis-mods/2022-11-06-120002
   _RetNet40_5x8_75_9_1_HPI-Docker/
```

6.9 Pipeline Finalisation Stage

1. Retrieving experiment results. Command for downloading all experiment's files

```
1 $ aws s3 sync s3://genesis-mods/2022-11-06-120002
   _RetNet40_5x8_75_9_1_HPI-Docker/
```

2. Retrieving experiment stats or results

```
1 $ aws s3 sync s3://genesis-mods/globalStatistics.txt
```

3. Data Analysis

6.10 Pipeline Cleanup Stage

1. Stopping container

```
1 $ docker ps
2 CONTAINER ID   IMAGE                                CREATED          NAMES
3 fde8447cad99   karolchlasta/genesis-sim:prod      21 hours ago    jovial_fermi
4
5 $ docker stop fde8447cad99
```

2. Removing Container from the local Store

```
1 $ docker ps -a
2 CONTAINER ID   IMAGE                                CREATED          NAMES
3 fde8447cad99   karolchlasta/genesis-sim:prod      21 hours ago    jovial_fermi
4
5 $ docker rm -f fde8447cad99
6 Deleted: sha256:8247
   b874067b83ea220baa3c6027c8b5ee69aa711b417257ebfdc47eab39e080
```

3. Removing NSP Container image. find image id to delete and run command to delete the image:

```
1 $ docker images
2 REPOSITORY          IMAGE ID        CREATED          SIZE
3 <none>              8247b874067b   4 weeks ago     294MB
4 <none>              2a22f5889924   4 weeks ago     63.1MB
5 karolchlasta/genesis-sim  db79b4ec632c   5 weeks ago     958MB
6
7 $ docker rmi -f db79b4ec632c
8 Deleted: sha256:8247
   b874067b83ea220baa3c6027c8b5ee69aa711b417257ebfdc47eab39e080
```

4. Data Analysis

References

- [1] T. Bekolay, J. Bergstra, E. Hunsberger, T. DeWolf, T. C. Stewart, D. Rasmussen, X. Choo, A. R. Voelker, and C. Eliasmith. “Nengo: a Python tool for building large-scale functional brain models”. In: *Frontiers in neuroinformatics* 7 (2014), page 48.
- [2] J. M. Bower, D. Beeman, and M. Hucka. “The GENESIS simulation system”. In: (2003).
- [3] G. Chen. “The rise of the enterprise container platform”. In: *IDC White Paper* (2018).
- [4] J. C. Crone, M. M. Vindiola, A. B. Yu, D. L. Boothe, D. Beeman, K. S. Oie, and P. J. Franaszczuk. “Enabling Large-Scale Simulations With the GENESIS Neuronal Simulator”. In: *Frontiers in neuroinformatics* 13 (2019), page 69.
- [5] I. C. Education. *Container Orchestration*. <https://www.ibm.com/cloud/learn/container-orchestration>, accessed on 27.06.2022. 2021.
- [6] A. L. Hodgkin and A. F. Huxley. “A quantitative description of membrane current and its application to conduction and excitation in nerve”. In: *The Journal of physiology* 117.4 (1952), pages 500–544.
- [7] W. A. Kamiński and G. M. Wójcik. “Liquid state machine built of Hodgkin-Huxley neurons-pattern recognition and informational entropy”. In: *Annales Universitatis Mariae Curie-Skłodowska, sectio AI-Informatica* 1.1 (2015), pages 1–7.
- [8] W. Maass. “Liquid state machines: motivation, theory, and applications”. In: *Computability in context: computation and logic in the real world* (2011), pages 275–296.
- [9] D. Merkel et al. “Docker: lightweight linux containers for consistent development and deployment”. In: *Linux j* 239.2 (2014), page 2.
- [10] J. Nickoloff and S. Kuenzli. *Docker in action*. Simon and Schuster, 2019.
- [11] C. Ramey. “Bash, the Bourne- Again Shell”. In: *Proceedings of The Romanian Open Systems Conference & Exhibition (ROSE 1994), The Romanian UNIX User’s Group (GURU)*. 1994, pages 3–5.
- [12] L. Wang, Z. Yang, S. Guo, L. Qu, X. Zhang, Z. Kang, and W. Xu. “LSMCore: A 69k-Synapse/mm² Single-Core Digital Neuromorphic Processor for Liquid State Machine”. In: *IEEE Transactions on Circuits and Systems I: Regular Papers* 69.5 (2022), pages 1976–1989.